

Teste de software de terceiros: Hotel Management System

- **Aluno:** Gabriel Felipe Jess Meira
- **Disciplina:** Verificação e Validação
- **Sistema sob teste (SUT):** [CrystalWang1225/Hotel_Management_System](https://github.com/CrystalWang1225/Hotel_Management_System), commit `71b396bab15a840deab61a05d5c762173e8ea410`
- **Fork com testes e correções:** https://github.com/gabrielfjm/Hotel_Management_System (tags `upstream-71b396b`, `sut-original`, `sut-corrigido`)
- **Ferramentas:** pytest 8.4.2, coverage.py via pytest-cov 6.3.0, Cosmic Ray 8.4.3, radon 6.0.1, pygount 3.1.0

Sumário executivo

As três técnicas foram aplicadas na ordem exigida (**funcional** → **estrutural** → **baseada em defeitos**) sobre as três funcionalidades centrais do sistema: realizar reserva, cancelar reserva e consultar disponibilidade.

Etap	Técnica	Código	Nº de casos	Cobertura de comandos	Cobertura de desvios	Escore de mutação
1	Funcional (classes de equivalência + valor limite)	original	52	93/103 (90,3%)	45/54 (83,3%)	—
2	Estrutural (meta: 100% de comandos e de desvios viáveis)	original	59	103/103 (100,0%)	53/54 (98,1%)	—
—	Correção dos 19 defeitos; mesma suíte	corrigida	59	106/106 (100,0%)	44/44 (100,0%)	191/204 (93,6%)
3	Baseada em defeitos (Cosmic Ray)	corrigida	64	106/106 (100,0%)	44/44 (100,0%)	196/204 (96,1%)

A cobertura considera as funções do recorte em `hotel/views.py` (seção 5). Os testes revelaram **19 defeitos** no código original. Entre eles, um predicado de conflito de datas que é uma tautologia: depois da primeira reserva, o quarto nunca mais pode ser reservado. Também há cancelamento de reservas de outros usuários e exclusão por requisição GET. Todos os defeitos foram corrigidos em commits separados no fork e confirmados pelos testes que os revelaram. A suíte final passa integralmente no programa corrigido. A mutação foi aplicada ao programa corrigido, que é o código submetido aos mutantes.

1. Seleção e caracterização do SUT

1.1 Origem, autoria e propósito

O *Hotel Management System* é uma aplicação web em Python de autoria de **Crystal Yuecen Wang**. Foi desenvolvida como projeto final da disciplina **ECE 464 – Databases**, sob orientação do professor Eugene Sokolov, e publicada no GitHub (8 commits, último em `71b396b`). Segundo o README, o objetivo é "*construir um sistema de gestão hoteleira eficiente e seguro, que ajude o negócio a se manter organizado e com as informações acessíveis*". O público são hóspedes, que se cadastram, reservam, alteram, cancelam e pagam, e a administração do hotel, que mantém quartos e tipos de quarto no banco.

Por que é adequado: é software de terceiros, escrito em Python e com finalidade real de uso. Tem banco relacional (6 tabelas com chaves estrangeiras), autenticação por sessão, regras de negócio com datas, capacidade e custo, e várias rotas HTTP. Isso justifica aplicar as três técnicas. O tamanho (≈350 linhas de código, seção 2) permite chegar a análises completas, como cobertura de desvios de 100% e mutação sem amostragem, em vez de análises parciais.

Arquitetura: Flask 0.12 (rotas em `hotel/views.py`), WTForms/Flask-WTF (`hotel/forms.py`), SQLAlchemy/Flask-SQLAlchemy 2.1 (`hotel/models.py`), SQLite e templates Jinja2 com Bootstrap. As relações são `reservations.ruid` → `user.uid`, `booked.brid` → `reservations.rid`, `booked.room_id` → `rooms.room_number` e `payment.prid` → `reservations.rid`.

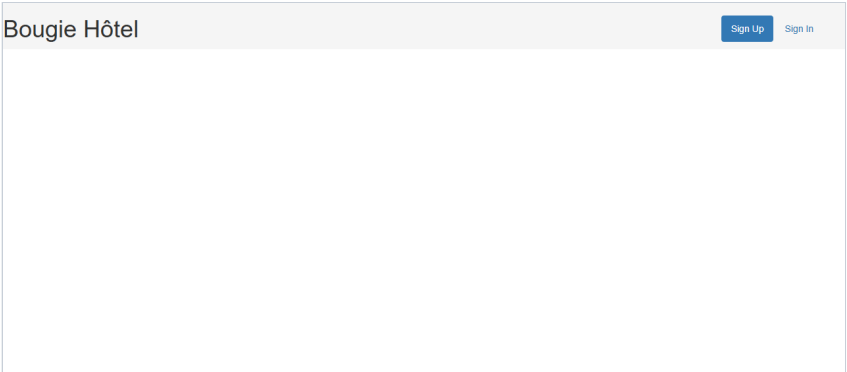
1.2 Funcionalidades

Funcionalidade	Rota	No recorte?
Cadastro de usuário	<code>/signup</code>	não
Login / logout por sessão	<code>/signin</code> , <code>/logout</code>	pré-condição
Listagem de quartos e tipos	<code>/rooms</code>	sim (REQ-03)
Consulta de disponibilidade por período e hóspedes	<code>/available</code> → <code>/rooms</code>	REQ-03
Reserva de um ou mais quartos, com validação de datas, ocupação e capacidade e cálculo do custo	<code>/reserve</code>	REQ-01
Minha conta: dados, reservas, custos e pagamentos	<code>/about_user</code>	apoio
Alteração de reserva	<code>/update/<rid></code>	não
Cancelamento de reserva	<code>/delete/<rid></code>	REQ-02
Pagamento com cartão	<code>/payment/<rid></code>	não (usado em CE-30)

O recorte cobre as três funcionalidades que concentram as regras de negócio descritas no README: datas no passado, quartos indisponíveis, capacidade insuficiente e custo por diária. Alteração e pagamento ficaram de fora por reaproveitarem as mesmas regras (alteração) ou por não terem regra verificável além da gravação (pagamento).

1.3 Evidência de instalação e execução

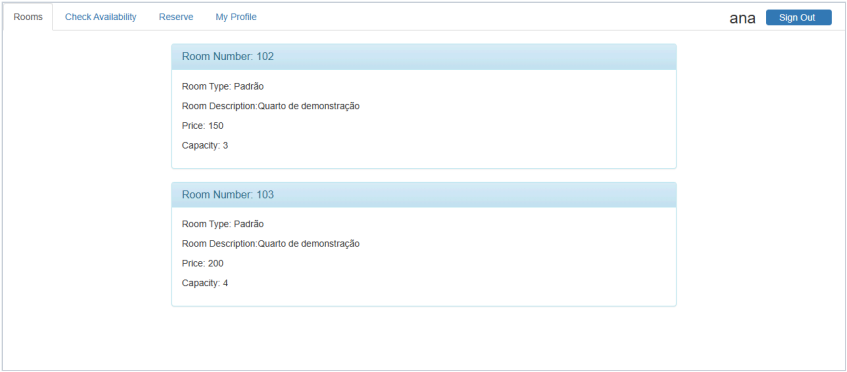
O sistema foi instalado com `setup_local.ps1`. O script cria um ambiente Python 3.9 com versões compatíveis com o código de 2019. `compat.py` repõe `time.clock`, removido no Python 3.8 e usado por Flask-SQLAlchemy 2.1. `hotel/__init__.py` passou a ler a URI do banco de `HOTEL_DB_URI`. **Nenhuma regra de negócio foi alterada nessa preparação:** o diff entre `upstream-71b396b` e `sut-original` não toca `views.py`, `models.py` nem `forms.py`. As telas abaixo foram capturadas pelo script `scripts/capturar_telas.py`, que sobe o servidor com a base de demonstração e navega com o Microsoft Edge.



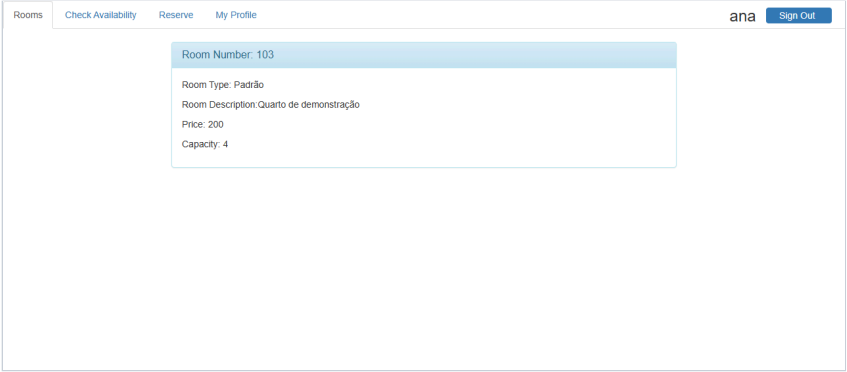
Página inicial do sistema original em execução (Flask, porta local).



Lista de quartos após o login de Ana (código original).



Consulta de D+20 a D+22 para 4 hóspedes no original: o 101, livre nesse período, não aparece (DEF-01), e o 102, com capacidade 3, aparece (DEF-14).



Mesma consulta no código corrigido: só o 103 comporta 4 hóspedes.

Rooms

Check Availability

Reservation

My Account

ana

Sign Out

The room you are reserving is not available at the time you selected!

Reserve Now

Reserva do 101 em D+20 a D+22 recusada no original, embora o quarto só esteja ocupado de D+10 a D+12 (DEF-01).

Rooms

Check Availability

Reserve

My Profile

ana

Sign Out

Room Number: 101

Room Type: Padrão

Room Description: Quarto de demonstração

Price: 100

Capacity: 2

Room Number: 102

Room Type: Padrão

Room Description: Quarto de demonstração

Price: 150

Capacity: 3

Room Number: 103

Room Type: Padrão

Mesma reserva aceita no código corrigido.

Rooms

Check Availability

Reservation

My Account

ana

Sign Out

First Name

Ana

Last Name

Silva

User Name

ana

Email

ana@example.test

Password

Reservation Number: 1

Room number: 101

Checkin-Date: 2026-10-10 00:00:00

Checkout-Date: 2026-10-12 00:00:00

Number of Guests: 2

Total Charge: \$200

Payments Status:

Reservation Number: 2

Room number: 101

Checkin-Date: 2026-10-20 00:00:00

Checkout-Date: 2026-10-22 00:00:00

Number of Guests: 2

Total Charge: \$200

Payments Status:

"My Account" no código corrigido: reservas, custos e o botão de cancelamento via POST.

2. Métricas de código

Medidas sobre o código do upstream (tag upstream-71b396b), extraído com `git archive`. Só entram `app.py` e o pacote `hotel/`. **Ficam de fora** os testes e scripts desta análise, os ambientes virtuais (`.venv*`) e as bibliotecas de front-end vendorizadas (`hotel/static/bootstrap`). Comando único: `python scripts/metricas_codigo.py`. As saídas completas estão em `evidencias/metricas/`.

Métrica	Valor	Ferramenta e comando
Módulos Python	5	<code>app.py</code> , <code>hotel/__init__.py</code> , <code>forms.py</code> , <code>models.py</code> , <code>views.py</code>
LOC (linhas físicas)	428	<code>radon raw -s app.py hotel</code>
SLOC (linhas de código)	354	<code>radon raw -s app.py hotel</code>
LLOC (linhas lógicas)	341	<code>radon raw -s app.py hotel</code>
Linhas de código (pygount)	352	<code>pygount --format=summary --suffix=py app.py hotel</code>
Classes	12	<code>radon cc -s -a app.py hotel</code> (6 formulários + 6 modelos)
Funções	13	idem (12 em <code>views.py</code> + <code>length_check</code>)
Métodos	6	idem (construtores dos modelos)
Complexidade ciclomática média / máxima	5,0 / 25	idem; <code>reserve</code> = 25 e <code>update_reservation</code> = 24 (grau D)

A diferença entre SLOC do radon (354) e Code do pygount (352) vem das duas linhas de docstring/continuação que cada ferramenta classifica de forma distinta. `reserve` (CC = 25) e `show_rooms` (CC = 16), as funções mais complexas do sistema, estão no recorte.

3. Metodologia

Oráculos. Os resultados esperados vêm do README ("check-in no passado", "quartos indisponíveis no período", "quartos que não comportam o número de hóspedes", "a reserva só pode ser alterada/excluída pelo usuário em *My Account*") e da interface, que traz os formulários com datas `MM/DD/AAAA`, hóspedes inteiros e quartos separados por vírgula. Para o que o README não fixa, foram adotadas convenções declaradas: estadia como intervalo semiaberto [`entrada`, `saída`), em que um hóspede pode entrar no dia em que outro sai; métodos HTTP seguros, em que `GET` não altera dados; e integridade referencial ao cancelar.

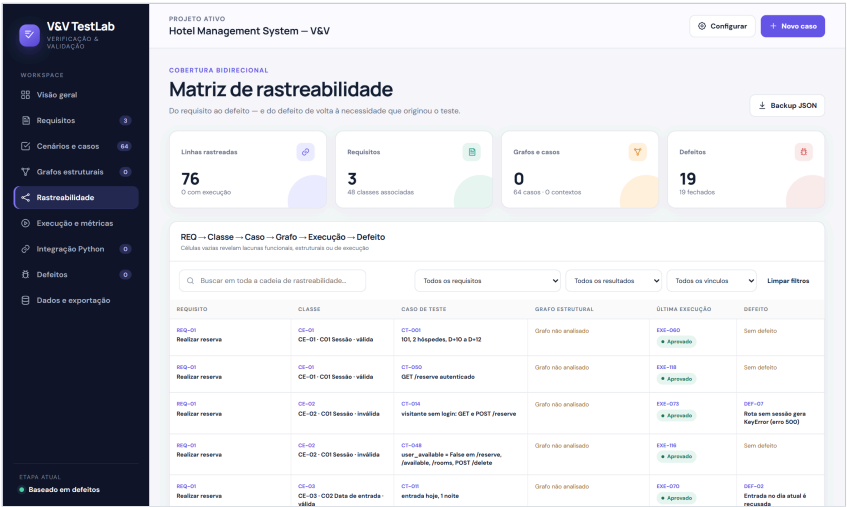
Ambiente. Windows 11, Python 3.9.25 (aplicação e pytest) e Python 3.12 (Cosmic Ray, em ambiente separado porque exige SQLAlchemy ≥ 1.4, incompatível com Flask-SQLAlchemy 2.1). Cada caso usa SQLite em memória recriado pela fixture `baseline`, com três quartos (101: R\$ 100/noite, 2 pessoas; 102: R\$ 150, 3; 103: R\$ 200, 4) e dois usuários (Ana e Bruno). As datas são relativas ao dia da execução, para manter a suíte reproduzível.

Versões do SUT. A variável `SUT_VERSION` escolhe o código importado pelos testes. Com `original`, é usado `.sut-original/hotel`, extraído da tag `sut-original`. Com corrigida, é usado `hotel/`. Na versão original, cada caso marcado `@pytest.mark.defeito("DEF-xx", ...)` recebe `xfail(strict=True)`: ele **precisa falhar** para confirmar que o defeito existe. Se um defeito "sumir", o `XPASS` estrito quebra a execução. Na versão corrigida, nenhum caso é marcado e todos precisam passar.

Rastreabilidade. Cada função de teste leva o identificador do caso no nome (`test_CT_001...`), a etapa em que foi criado (`@pytest.mark.funcional`, `estrutural` ou `mutacao`), as classes que exercita (`@pytest.mark.ce("CE-01", ...)`) e, quando é o caso, o defeito revelado. O plugin em `tests/conftest.py` exporta essa matriz (`--rastreabilidade`), e `scripts/evolucao.py` verifica que **toda classe de equivalência tem ao menos um caso funcional** (Apêndice A). A cadeia é REQ → CE → CT → execução pytest → DEF → commit de correção.

Execução. `etapas.ps1` roda tudo em sequência e grava as evidências em `evidencias/<etapa>/:` `pytest.txt` (saída `-v -rxX` e relatório `term-missing`), `junit.xml`, `coverage.json`, `htmlcov/`, `rastreabilidade.json` e `cobertura-recorte.json`. A ordem das etapas também aparece no histórico do fork (seção 6).

Gestão dos testes (V&V TestLab). Requisitos, classes, casos, execuções, defeitos e métricas também foram organizados no V&V TestLab, uma aplicação web local de gestão de testes. Ela se conecta ao repositório por uma ponte HTTP restrita a `127.0.0.1` (`integration/vv_bridge.py`), que executa o pytest com `coverage.py` e o Cosmic Ray e importa o JUnit, a cobertura das funções do recorte e o escore de mutação, vinculando cada resultado ao caso pelo identificador `CT-xxx`. O projeto importável (`output/hotel-vvtestlab-projeto-inicial.json`) e o estudo completo (`output/hotel-vvtestlab-backup.json`) são gerados das mesmas evidências deste relatório.



Matriz de rastreabilidade do V&V TestLab com o estudo do hotel carregado.

4. Etapa 1: teste funcional

4.1 Classes de equivalência

As 22 condições de entrada das três funcionalidades foram particionadas em **48 classes** (31 válidas e 17 inválidas). As condições C10 e C19 (formato das datas) e a classe CE-45/CE-48 (hóspedes não inteiros) foram acrescentadas na revisão da tabela (commit `8bf46d8`), antes das correções. Os IDs CE-43 a CE-48 refletem essa inclusão.

Req.	Condição de entrada	Classes válidas	Classes inválidas
REQ-01 Reserva	C01 Sessão	CE-01 autenticado	CE-02 sem sessão / sessão encerrada
	C02 Data de entrada	CE-03 hoje ou futura	CE-04 passada
	C03 Duração (saída – entrada)	CE-05 ≥ 1 noite	CE-06 0 noites; CE-07 negativa
	C04 N° de hóspedes	CE-08 inteiro de 1 até a capacidade somada	CE-09 < 1; CE-10 > capacidade; CE-45 não inteiro
	C05 Formato dos quartos	CE-11 inteiros separados por vírgula	CE-12 texto não numérico
	C06 Existência dos quartos	CE-13 todos existem	CE-14 algum não existe
	C07 Repetição	CE-15 sem repetição	CE-16 quarto repetido
	C08 Ocupação no período	CE-17 sem interseção (inclui adjacentes)	CE-18 interseção de ≥ 1 noite
	C09 Quantidade de quartos	CE-19 um; CE-20 vários	—
	C10 Formato das datas	CE-43 MM/DD/AAAA válido	CE-44 malformada

Req.	Condição de entrada	Classes válidas	Classes inválidas
REQ-02 Cancelamento	C11 Sessão	CE-21 autenticado	CE-22 sem sessão
	C12 Existência da reserva	CE-23 existente	CE-24 inexistente
	C13 Titularidade	CE-25 própria	CE-26 de outro usuário
	C14 Forma da requisição	CE-27 confirmação explícita (POST)	CE-28 navegação por link (GET)
	C15 Registros dependentes	CE-29 sem pagamento; CE-30 com pagamento	—
REQ-03 Disponibilidade	C16 Sessão	CE-31 autenticado	CE-32 sem sessão
	C17 Filtro	CE-33 sem filtro (todos); CE-34 com período e hóspedes	—
	C18 Período	CE-35 entrada < saída	CE-36 entrada ≥ saída
	C19 Formato das datas	CE-46 válido	CE-47 malformada
	C20 Nº de hóspedes	CE-37 inteiro ≥ 1	CE-38 < 1; CE-48 não inteiro
	C21 Ocupação de cada quarto	CE-39 livre → exibido; CE-40 ocupado → omitido	—
	C22 Capacidade de cada quarto	CE-41 capacidade ≥ hóspedes → exibido; CE-42 capacidade < hóspedes → omitido	—

Regra de derivação. Um caso pode cobrir várias classes válidas ao mesmo tempo (CT-001 cobre dez). Cada classe inválida tem ao menos um caso próprio, em que **todas as outras entradas são válidas**, para que a rejeição só possa ser atribuída àquela classe. Por isso o CT-016 usa `101,999` com 2 hóspedes, e não `999` com 0 hóspedes, o que misturaria CE-14 e CE-09.

4.2 Análise do valor limite

Datas relativas a hoje (D). Reserva existente usada nos limites de interseção: quarto 101 de D+10 a D+12.

Variável (limite)	Imediatamente abaixo	No limite	Imediatamente acima	Justificativa
Data de entrada (mín. = hoje)	D−1 → rejeitar (CT-013)	D → aceitar (CT-011)	D+1 → aceitar (CT-012)	O README exige "pelo menos hoje"; a fronteira passado/presente é onde se costuma errar comparando data com data-hora.
Duração da reserva (mín. = 1 noite)	0 → rejeitar (CT-009)	1 → aceitar, custo 100 (CT-008)	2 → aceitar, custo 200 (CT-001)	Saída = entrada não gera diária; −1 (CT-010) representa CE-07.
Hóspedes, mínimo (1)	0 → rejeitar (CT-007)	1 → aceitar (CT-006)	2 → aceitar (CT-003)	Uma reserva precisa de ao menos um hóspede.
Hóspedes, máximo de 1 quarto (101: 2)	1 → aceitar (CT-006)	2 → aceitar (CT-003)	3 → rejeitar (CT-004)	A capacidade é o limite superior de CE-08.
Hóspedes, máximo somado (101+102: 5)	4 → aceitar (CT-060)	5 → aceitar (CT-002)	6 → rejeitar (CT-005)	Verifica se a capacidade é somada quando há vários quartos.
Fim da nova estadia × entrada existente (D+10)	saída D+11 → conflito (CT-019)	saída D+10 → livre (CT-021)	saída D+9 → livre (CT-061)	Com intervalo [entrada, saída), sair no dia em que o outro entra não conflita.
Início da nova estadia × saída existente (D+12)	entrada D+11 → conflito (CT-020)	entrada D+12 → livre (CT-022)	entrada D+13 → livre (CT-062)	Simétrico ao anterior.
Período da consulta (mín. = 1 noite)	0 → recusar (CT-038)	1 → aceitar (CT-043)	2 → aceitar (CT-032)	Mesma regra da reserva; −2 (CT-037) representa entrada > saída.
Hóspedes na consulta (mín. 1)	0 → recusar (CT-039)	1 → aceitar (CT-042)	2 → aceitar (CT-040)	—
Capacidade do 101 (2) na consulta	1 → exibe 101 (CT-042)	2 → exibe 101 (CT-040)	3 → omite 101 (CT-041)	Fronteira de CE-41/CE-42.
Interseção na consulta	D+11 a D+13 → omite (CT-036)	D+12 a D+14 → exibe (CT-034); D+8 a D+10 → exibe (CT-035)	D+13 a D+15 → exibe (CT-064); D+7 a D+9 → exibe (CT-063)	Mesmos limites da reserva, agora no filtro.

4.3 Casos funcionais e resultados no código original

Legenda: **P** = passou no original; **F** = falhou no original, confirmando o defeito indicado (xfail estrito). No código corrigido, todos passam.

Caso	Classes	Entrada / cenário	Resultado esperado	Original
CT-001	CE-01,03,05,08,11,13,15,17,19,43	101, 2 hóspedes, D+10 a D+12	redireciona a <code>/rooms</code> ; custo 200; vínculo (rid, 101)	P
CT-002	CE-08, CE-20	101,102; 5 hóspedes; 3 noites	aceita; custo (100+150)×3 = 750	P
CT-003	CE-08	101 com 2 hóspedes (= capacidade)	aceita	P
CT-004	CE-10	101 com 3 hóspedes	recusa (<code>/reserve</code>), nada gravado	P
CT-005	CE-10	101,102 com 6 hóspedes (capacidade 5)	recusa	P
CT-006	CE-08	1 hóspede	aceita	P
CT-007	CE-09	0 hóspedes	recusa	F – DEF-03
CT-008	CE-05	1 noite	aceita; custo 100	P
CT-009	CE-06	saída = entrada	recusa	P
CT-010	CE-07	saída = entrada − 1	recusa	P
CT-011	CE-03	entrada hoje, 1 noite	aceita	F – DEF-02
CT-012	CE-03	entrada amanhã	aceita	P
CT-013	CE-04	entrada ontem	recusa	P
CT-014	CE-02	visitante sem login: GET e POST <code>/reserve</code>	redireciona a <code>/</code> ; nada gravado	F – DEF-07
CT-015	CE-12	quartos <code>abc</code>	recusa	F – DEF-04
CT-016	CE-14	quartos <code>101,999</code> , 2 hóspedes	recusa; nenhum vínculo	F – DEF-05
CT-017	CE-16	quartos <code>101,101</code>	recusa	F – DEF-06
CT-018	CE-18	101 já reservado em D+10–12; nova D+10–12	recusa	P

Caso	Classes	Entrada / cenário	Resultado esperado	Original
CT-019	CE-18	idem; nova D+9–11	recusa	P
CT-020	CE-18	idem; nova D+11–13	recusa	P
CT-021	CE-17	idem; nova D+8–10	aceita	F – DEF-01
CT-022	CE-17	idem; nova D+12–14	aceita	F – DEF-01
CT-023	CE-17	idem; nova D+20–22	aceita	F – DEF-01
CT-024	CE-21,23,25,27,29	Ana cancela a própria reserva (POST)	/rooms; reserva e vínculo removidos	P
CT-025	CE-22	visitante sem login POST /delete/<rid>	redireciona; reserva mantida	F – DEF-07
CT-026	CE-24	POST /delete/999	404	F – DEF-09
CT-027	CE-26	Bruno cancela reserva de Ana	403; reserva e vínculo mantidos	F – DEF-08
CT-028	CE-28	GET /delete/<rid>	405; reserva mantida	F – DEF-10
CT-029	CE-21,23,25,27,30	cancelar reserva com pagamento	reserva e pagamento removidos	F – DEF-11
CT-030	CE-31, CE-33	/rooms sem filtro, com 101 reservado	lista 101, 102, 103	P
CT-031	CE-32	visitante sem login: /available (GET/POST) e /rooms	redireciona a /	F – DEF-07
CT-032	CE-31,34,35,37,39,40,41,46	101 ocupado D+10–12; consulta D+10–12, 2 hóspedes	lista 102, 103	P
CT-033	CE-39	101 reservado em D+20; consulta D+10–12	lista 101, 102, 103	F – DEF-01
CT-034	CE-39	101 ocupado D+10–12; consulta D+12–14	lista 101, 102, 103	F – DEF-01
CT-035	CE-39	idem; consulta D+8–10	lista 101, 102, 103	F – DEF-01
CT-036	CE-40	idem; consulta D+11–13	lista 102, 103	P
CT-037	CE-36	consulta com entrada D+12 e saída D+10	recusa (/available)	F – DEF-12
CT-038	CE-36	consulta com entrada = saída	recusa	F – DEF-12
CT-039	CE-38	consulta com 0 hóspedes	recusa	F – DEF-13
CT-040	CE-41	consulta com 2 hóspedes	lista 101, 102, 103	P
CT-041	CE-42	consulta com 3 hóspedes	lista 102, 103	F – DEF-14
CT-042	CE-37, CE-41	consulta com 1 hóspede	lista 101, 102, 103	P
CT-043	CE-35	consulta de 1 noite	aceita; lista todos	P
CT-051	CE-44	reserva com entrada 2026-13-01	recusa	F – DEF-18
CT-052	CE-45	reserva com hóspedes dois	recusa	F – DEF-19
CT-053	CE-47	consulta com entrada amanhã	recusa; lista sem filtro	F – DEF-18
CT-054	CE-48	consulta com hóspedes dois	recusa	F – DEF-13
CT-060	CE-08, CE-20	101,102; 4 hóspedes (um abaixo da capacidade 5); 1 noite	aceita; custo 250	P
CT-061	CE-17	101 reservado D+10–12; nova D+7–9	aceita	F – DEF-01
CT-062	CE-17	idem; nova D+13–15	aceita	F – DEF-01
CT-063	CE-39	101 ocupado D+10–12; consulta D+7–9	lista 101, 102, 103	F – DEF-01
CT-064	CE-39	idem; consulta D+13–15	lista 101, 102, 103	F – DEF-01

Resultado (código original, evidencias/functional-original/): 52 casos, **22 passaram e 30 falharam** confirmando 16 defeitos (DEF-01 a DEF-14, DEF-18 e DEF-19). As falhas têm causas distintas, conferidas com `--runxfail: KeyError: 'user_available', ValueError em int('abc'), UnmappedInstanceError, TypeError em combine(None)`, além de redirecionamentos e listagens diferentes do oráculo. Cobertura do recorte: **93/103 comandos (90,3%) e 45/54 desvios (83,3%)**. No `views.py` inteiro, a cobertura é de 53%, porque as rotas fora do recorte não são exercitadas.

5. Etapa 2: teste estrutural

5.1 Meta de cobertura e justificativa

Critério: todos-os-nós e todos-os-arcos (comandos e desvios, `coverage run --branch`) nas cinco funções do recorte: `reserve`, `cal_cost`, `delete_reservation`, `check_available` e `show_rooms`, com 103 comandos e 54 desvios no original. **Meta: 100% dos comandos e 100% dos desvios viáveis.**

Justificativa:

1. O recorte é pequeno e concentra toda a regra de negócio testada, então a meta máxima tem custo baixo.
2. Cobertura de desvios subsume a de comandos e obriga a exercitar os ramos falsos (sessão encerrada, laço sem conflito), justamente onde a etapa funcional deixou lacunas.
3. Critérios mais fortes, como caminhos simples, explodem combinatoriamente por causa dos laços aninhados de `reserve/show_rooms` (três níveis) e trariam pouco ganho diante do teste de mutação da etapa seguinte.

A medição é restrita ao recorte porque medir `views.py` inteiro misturaria rotas deliberadamente não testadas (cadastro, login, alteração, pagamento) e tornaria qualquer meta global arbitrária.

5.2 Lacunas após a etapa funcional e casos acrescentados

Trecho não coberto (original)	Por que ficou de fora	Caso estrutural
<code>delete_reservation</code> 88–89, <code>show_rooms</code> 166–167, <code>check_available</code> 180–181, <code>reserve</code> 242–243 (ramo falso de <code>if session['user_available']</code>)	Os casos funcionais sem sessão não criam a chave, e o original lança <code>KeyError</code> antes do <code>if</code> ; o ramo falso só roda com a chave valendo <code>False</code> , estado deixado pelo <code>logout</code>	CT-048
<code>check_available</code> 179 (desvio 175–179)	A etapa funcional só fazia <code>POST</code>	CT-049
<code>reserve</code> 241 (desvio 189–241)	Idem	CT-050

Trecho não coberto (original)	Por que ficou de fora	Caso estrutural
<code>reserve</code> desvios 208→205 e 205→204 (vínculo de outro quarto; fim do laço interno sem conflito)	Nos casos funcionais, todas as reservas prévias eram do mesmo quarto	CT-044
<code>reserve</code> desvio 230→229 (<code>if each.rid > current_id</code> falso)	Inviável: os IDs vêm em ordem crescente de chave primária, então cada <code>rid</code> supera o máximo anterior	—

A leitura do código para essas lacunas expôs mais três defeitos, que ganharam casos próprios: CT-045 (DEF-15, laço triplo sem `brid == rid`), CT-046 (DEF-16, variável global `global_avail`) e CT-047 (DEF-17, `remove` durante a iteração).

Caso	Classes	Cenário	Esperado	Original
CT-044	CE-17, CE-19	Bruno tem o 102 em D+10; Ana reserva o 101 em D+10	aceita	passou
CT-045	CE-17	Ana tem o 101 em D+10, Bruno tem o 102 em D+20; reservar o 101 em D+20	aceita	falhou – DEF-15 (mascarado por DEF-01)
CT-046	CE-33	Ana consulta D+10 com o 101 ocupado; Bruno, em outra sessão, abre <code>/rooms</code> sem filtro	Bruno vê 101, 102, 103	falhou – DEF-16 (vê 102, 103)
CT-047	CE-40	101 e 102 ocupados em D+10; consulta D+10, 1 hóspede	só o 103	falhou – DEF-17 (102 aparece)
CT-048	CE-02, CE-22, CE-32	<code>user_available = False</code> em <code>/reserve</code> , <code>/available</code> , <code>/rooms</code> , POST <code>/delete</code>	redireciona e preserva	passou
CT-049	CE-31	GET <code>/available</code> autenticado	formulário com <code>checkin_date</code>	passou
CT-050	CE-01	GET <code>/reserve</code> autenticado	formulário com <code>room_numbers</code>	passou

Resultado (código original, `evidencias/estrutural-original/`): 59 casos, 26 passaram e 33 falharam (xfail estrito). Cobertura do recorte: **103/103 comandos (100%) e 53/54 desvios (98,1%)**. A meta foi atingida, porque o desvio restante é inviável.

6. Correção dos defeitos antes da mutação

A mutação precisa de uma suíte que passe no programa mutado. Por isso os defeitos foram corrigidos **antes** de gerar os mutantes, em commits separados no fork, cada um citando a causa e os casos que o confirmam. O diff completo está em `evidencias/correcoes.diff` (`git diff sut-original sut-corrigido -- hotel`).

Commit	Defeitos	Casos que passaram a passar
8a6c4bd	DEF-07 (sessão sem chave → <code>KeyError</code>)	CT-014, CT-025, CT-031
2d1b2cd	DEF-01 (tautologia), DEF-15 (produto cartesiano)	CT-021, 022, 023, 033, 034, 035, 061–064, 045
704bc86	DEF-02 (entrada hoje)	CT-011
644fd0d	DEF-03, 04, 05, 06, 18, 19 (validação da reserva; transação única)	CT-007, 015, 016, 017, 051, 052
1313f12	DEF-08, 09, 10, 11 (cancelamento; template com POST)	CT-026, 027, 028, 029
9e66e2d	DEF-12, 13, 14, 16, 17, 18 (consulta; filtro na sessão)	CT-037, 038, 039, 041, 053, 054, 046, 047

Em cada passo, a suíte inteira foi executada para confirmar que nenhum caso que já passava regrediu. **Resultado final no programa corrigido (tag `sut-corrigido`, `evidencias/suíte-corrigida/`):** **59/59 casos passaram**, sem `skip` nem `xfail`, com cobertura do recorte de **106/106 comandos e 44/44 desvios (100%)**. O código corrigido tem menos desvios que o original (44 contra 54) porque os laços triplos foram trocados por uma junção SQL e por compreensões de conjunto. As funções auxiliares (`_sessao_autenticada`, `_periodos_conflitam`, `_quartos_ocupados`, `_ler_quartos`) entram no recorte medido.

Um ajuste de oráculo foi necessário e está registrado no commit `8a6c4bd`. O CT-025 esperava que o cancelamento sem sessão redirecionasse para `/`, mas o sistema redireciona para `/rooms`, que exige login e então leva a `/`. Isso não é defeito do software: o requisito verificado, preservar a reserva, não mudou.

7. Etapa 3: teste baseado em defeitos (mutação)

7.1 Configuração e escopo

- **Ferramenta:** Cosmic Ray 8.4.3, distribuidor local, timeout de 30 s por mutante e todos os operadores padrão: substituição de operadores relacionais, aritméticos, lógicos e unários, troca de números, de `break/continue` e de palavras-chave, laço de zero iterações, remoção de decorador etc.
- **Escopo:** **todos os mutantes** gerados nas linhas das funções do recorte do `hotel/views.py` corrigido, sem amostragem: `reserve`, `cal_cost`, `delete_reservation`, `check_available`, `show_rooms` e as quatro auxiliares. Total: **204 mutantes**. As rotas fora do recorte foram excluídas porque não têm testes por decisão de escopo; incluí-las só acrescentaria sobreviventes triviais.
- **Configuração:** `mutacao/cosmic-ray-inicial.toml` e `mutacao/cosmic-ray-final.toml`. O comando de teste é `pytest -x -q`, restrito a `-m 'funcional or estrutural'` na rodada inicial. O script `scripts/mutacao.py` monta a sessão, executa `cosmic-ray exec` e grava `sessao.sqlite`, `resumo.json`, `sobreviventes.txt` (diff de cada sobrevivente) e `execucao.txt`.
- **Score:** mortos / (mortos + sobreviventes). Não houve mutantes incompetentes nem `timeouts`.

7.2 Rodada inicial (suíte ao fim da etapa estrutural)

204 mutantes: 191 mortos e 13 sobreviventes. Score de 93,6%. Execução em cerca de 4,5 minutos.

#	Função (linha)	Mutação	Classificação	Ação
S1	<code>_quartos_ocupados</code> (23)	<code>Booked.brid == Reservations.rid</code> → <code>>=</code>	Não equivalente. Nenhum caso tinha um vínculo de reserva <i>posterior</i> capaz de herdar as datas de uma reserva <i>anterior</i> de outro quarto	CT-055
S2	<code>_ler_quartos</code> (34)	<code>set(numeros) <= existentes</code> → <code><</code>	Não equivalente. Nenhum caso reservava <i>todos</i> os quartos do hotel	CT-056

#	Função (linha)	Mutação	Classificação	Ação
S3	<code>delete_reservation</code> (113)	<code>ruid != us.uid → is not</code>	Não equivalente. Só coincide para inteiros de -5 a 256, que o CPython mantém em cache; os IDs de usuário da base de teste eram 1 e 2	CT-057 (uid 1000)
S4	<code>delete_reservation</code> (113)	<code>ruid != us.uid → <</code>	Não equivalente. O CT-027 testa apenas Bruno (uid 2) cancelando reserva de Ana (uid 1), sentido em que <code><</code> também nega	CT-058
S5	<code>cal_cost</code> (268)	<code>each_room_id == e.room_number → is</code>	Não equivalente. Mesmo motivo de S3; os quartos da base eram 101–103	CT-059 (quarto 301)
S6	<code>check_available</code> (197)	<code>request.method == 'POST' → >=</code>	Equivalente. A rota só admite <code>GET</code> , <code>POST</code> , <code>HEAD</code> e <code>OPTIONS</code> , e nenhum método diferente de <code>POST</code> é <code>≥ 'POST'</code> na ordem lexicográfica	—
S7	<code>reserve</code> (217)	<code>idem</code>	Equivalente (mesmo raciocínio)	—
S8	<code>reserve</code> (226)	<code>time(0, 0) → time(1, 0)</code> em <code>d1</code>	Equivalente. <code>d1</code> só é comparado com datas à meia-noite (<code>d2</code> e datas gravadas) e com <code>date.today()</code> via <code>d1.date()</code> ; deslocar a hora de <code>d1</code> dentro do mesmo dia não inverte nenhuma comparação, e o valor gravado é <code>checkin</code> , não <code>d1</code>	—
S9	<code>reserve</code> (226)	<code>time(0, 0) → time(0, 1)</code>	Equivalente (idem)	—
S10	<code>reserve</code> (247)	custo inicial <code>0 → 1</code>	Equivalente. O valor é sobrescrito por <code>cal_cost</code> antes do único <code>commit</code>	—
S11	<code>reserve</code> (247)	custo inicial <code>0 → -1</code>	Equivalente (idem)	—
S12	<code>_ler_quartos</code> (34)	<code>len(set(n)) != len(n) → <</code>	Equivalente. Um conjunto nunca tem mais elementos que a lista de origem, então <code>!=</code> e <code><</code> são a mesma condição	—
S13	<code>_ler_quartos</code> (34)	<code>len(set(n)) != len(n) → is not</code>	Equivalente no domínio. Os comprimentos só passam de 256 se o usuário informar mais de 256 quartos numa reserva, o que é irrealista; abaixo disso, o cache de inteiros do CPython torna <code>is not</code> idêntico a <code>!=</code>	—

7.3 Casos acrescentados e rodada final

Caso	Classes	Cenário	Esperado	Mata
CT-055	CE-17	Ana tem o 102 em D+10 (reserva 1); Bruno tem o 101 em D+20 (reserva 2); Ana reserva o 101 em D+10	aceita	S1
CT-056	CE-13, CE-20	Reservar 101, 102 e 103 para 9 hóspedes por 2 noites	aceita; custo 900; 3 vínculos	S2
CT-057	CE-21, 23, 25, 27	Usuária com uid 1000 cancela a própria reserva	cancelada	S3
CT-058	CE-26	Ana (uid 1) tenta cancelar reserva de Bruno (uid 2)	403; reserva mantida	S4
CT-059	CE-05, CE-19	Quarto 301 (R\$ 120) por 2 noites	custo 240	S5

Rodada final (suíte completa, 64 casos, evidencias/mutacao-final/): 204 mutantes: 196 mortos, 8 sobreviventes, 0 incompetentes. Escore de 96,1% (inicial: 93,6%), em 271 s. Os cinco casos novos mataram os cinco sobreviventes não equivalentes. Restam 8 sobreviventes, todos classificados como equivalentes (S6 a S13); não equivalentes restantes: 0. O arquivo `evidencias/mutacao-final/sobreviventes.txt` traz o diff de cada um.

7.4 Fragilidades reveladas

A suíte tinha 100% de cobertura de desvios e, mesmo assim, deixou vivos 5 mutantes não equivalentes. As fragilidades expostas foram:

- Dados de teste homogêneos.** Todos os IDs e números de quarto cabiam no cache de inteiros pequenos do Python, o que esconde a troca de `==` por `is`. Em produção, com mais de 256 usuários ou quartos numerados como 301, esse erro passaria despercebido.
- Oráculos assimétricos.** A autorização foi testada só num sentido (uid maior tentando cancelar reserva de uid menor).
- Cenários de ocupação sempre com a mesma ordem de criação.** Faltava o caso em que a reserva mais nova pertence ao quarto pedido e a mais antiga a outro quarto.
- Nenhum caso no extremo superior da enumeração de quartos,** isto é, reservar todos eles.

Os 8 equivalentes vêm de código defensivo ou redundante que veio do original: o custo provisório 0 e `time(0, 0)` explícito. Eles mostram também que o escore bruto subestima a qualidade da suíte. Excluindo os equivalentes, o escore final é 100,0% (196/196).

8. Evolução incremental

Etap	Técnica	Código	Nº de casos	Passaram	Falharam (defeito confirmado)	Cobertura de comandos	Cobertura de desvios	Escore de mutação
1. Funcional	Classes de equivalência + valor limite	original	52	22	30	93/103 (90,3%)	45/54 (83,3%)	—
2. Estrutural	Cobertura de comandos e desvios	original	59	26	33	103/103 (100,0%)	53/54 (98,1%)	—
3. Correção	Suíte funcional + estrutural no código corrigido	corrigida	59	59	0	106/106 (100,0%)	44/44 (100,0%)	191/204 (93,6%)
4. Mutação	Casos para matar mutantes sobreviventes	corrigida	64	64	0	106/106 (100,0%)	44/44 (100,0%)	196/204 (96,1%)

Na linha 3, o escore é o da rodada **inicial** do Cosmic Ray, executada com a suíte funcional + estrutural sobre o código corrigido. Na linha 4, é o da rodada **final**, com a suíte completa. Nas etapas 1 e 2 não há escore, pois a mutação só é aplicada depois da correção dos defeitos (seção 6). Fonte: `evidencias/evolucao.json`, gerado por `scripts/evolucao.py`.

A ordem **Funcional** → **Estrutural** → **(correção)** → **Mutação** está no histórico do fork: `afeb312` (etapa 1), `ac975b7` (etapa 2), seis commits de correção, e `324e29a` (etapa 3). Cada etapa só acrescenta casos. Nenhum caso das etapas anteriores foi removido ou marcado com `skip`.

9. Registro de defeitos

Linhas referentes ao `hotel/views.py` original (`sut-original`). Todos os defeitos têm teste automatizado que falha no original e passa no corrigido.

ID	Gravidade	Função (linhas)	Entrada usada	Esperado	Obtido no original	Teste(s)	Causa no código	Correção (commit)
DEF-01	Crítica	reserve 208–210; show_rooms 160–162	101 reservado D+10–12; reservar 101 em D+20–22	aceita	recusado: "not available"	CT-021, 022, 023, 033, 034, 035, 061–064	(c1<=d1 and d2<=c2) or (d1<=c1 and d2<=c2) or (c1<=d1 and c2<=d2) or (d1<=c1 and c2<=d2) é tautologia: para quaisquer datas, uma alternativa é verdadeira. Quarto reservado uma vez fica bloqueado para sempre	_periodos_conflictam(a1, a2, b1, b2) = a1 < b2 and b1 < a2 (2d1b2cd)
DEF-02	Média	reserve 199	entrada hoje, 1 noite	aceita	recusado: "at least today"	CT-011	d1 (hoje 00:00) comparado com datetime.now()	d1.date() < date.today() (704bc86)
DEF-03	Alta	reserve 217	101, 0 hóspedes	recusa	reserva gravada com 0 hóspedes e custo 200	CT-007	só há limite superior num > total_num	exigir num >= 1 antes de gravar (644fd0d)
DEF-04	Média	reserve 208, 215	quartos abc	recusa com mensagem	500 ValueError: invalid literal for int()	CT-015	int(each) sem tratamento	_ler_quartos converte com try/except (644fd0d)
DEF-05	Alta	reserve 213–216, 233–236	quartos 101, 999, 2 hóspedes	recusa	gravado; vínculo com quarto 999 inexistente	CT-016	a existência só influencia indiretamente a soma de capacidades	_ler_quartos exige set(numeros) <= existentes (644fd0d)
DEF-06	Alta	reserve 213–216, 233–236	quartos 101, 101, 2 hóspedes	recusa	2 vínculos, capacidade 4 e custo 400 (dobro)	CT-017	lista não é deduplicada nem validada	_ler_quartos recusa repetição (644fd0d)
DEF-07	Média	delete_reservation 79; show_rooms 144; check_available 172; reserve 186	visitante sem login abre /reserve	redirecionar ao início	500 KeyError: 'user_available'	CT-014, 025, 031	session['user_available'] sem a chave, que só existe após login/logout	_sessao_autenticada() com session.get(..., False) (8a6c4bd)
DEF-08	Crítica	delete_reservation 79–86	Bruno envia POST /delete/<rid de Ana>	403, reserva mantida	reserva de Ana excluída	CT-027 (e CT-058)	só verifica se <i>alguém</i> está logado, não o titular	403 se reserva.ruid != usuário.uid (1313f12)
DEF-09	Média	delete_reservation 80–81	POST /delete/999	404	500 UnmappedInstanceError	CT-026	query.get devolve None e db.session.delete(None) falha	abort(404) (1313f12)
DEF-10	Crítica	rota /delete/<rid> (77); about_user.html	GET /delete/<rid>	405, nada alterado	reserva excluída	CT-028	exclusão por GET: links, pré-carregamento do navegador ou robôs apagam dados	rota só POST; lixeira vira formulário POST com confirmação (1313f12)
DEF-11	Alta	delete_reservation 81–86	cancelar reserva com pagamento	nenhum registro órfão	payment.prid aponta para reserva inexistente	CT-029	exclusão ignora Payment e usa vários commit()	apaga Booked e Payment junto, em uma transação (1313f12)
DEF-12	Média	check_available 175–178; show_rooms 162	consulta com entrada D+12, saída D+10	recusar com mensagem	aceita e lista todos os quartos, sem aviso	CT-037, 038	nenhum teste do período; show_rooms ignora o filtro em silêncio com c_in < c_out	valida checkout > checkin (9e66e2d)
DEF-13	Média	check_available 175–178	consulta com 0 ou dois hóspedes	recusar	aceita	CT-039, 054	num_guests não é validado	exige inteiro >= 1 (9e66e2d)
DEF-14	Média	show_rooms 155–163	consulta com 3 hóspedes	omitir o 101 (capacidade 2)	101 listado	CT-041	o número de hóspedes informado nunca é usado	filtra capacity >= hóspedes (9e66e2d)
DEF-15	Alta	reserve 203–210; show_rooms 155–160	101 de Ana em D+10, 102 de Bruno em D+20; reservar 101 em D+20	aceita	recusado (vínculo do 101 × datas da reserva do 102)	CT-045 (e CT-055)	laço triplo reservas × vínculos sem brid == rid; mascarado por DEF-01	junção Booked.brid == Reservations.rid em _quartos_ocupados (2d1b2cd)
DEF-16	Alta	global_avail 140; check_available 176–177; show_rooms 149–164	Ana consulta D+10; Bruno abre /rooms	Bruno vê todos os quartos	Bruno vê a lista filtrada de Ana	CT-046	filtro guardado em variável de módulo compartilhada por todas as sessões	filtro em session['filtro_disponibilidade'] (9e66e2d)
DEF-17	Alta	show_rooms 163	101 e 102 ocupados; consulta 1 hóspede	só o 103	102 aparece	CT-047	all_rooms.remove(each) dentro de for each in all_rooms pula o elemento seguinte	lista nova por compreensão (9e66e2d)
DEF-18	Média	reserve 191–192; show_rooms 153–154	data 2026-13-01	recusar	500 TypeError: combine() argument 1 must be datetime.date, not None	CT-051, 053	DateTimeField inválido deixa data = None	valida datas antes de usar (644fd0d, 9e66e2d)
DEF-19	Média	reserve 217	hóspedes dois	recusar	500 TypeError: '>' not supported	CT-052	IntegerField inválido deixa None	exige inteiro >= 1 (644fd0d)

Fora do recorte: update_reservation, about_user e payment repetem a causa de DEF-07 (session['user_available']), e update_reservation repete a de DEF-01 e DEF-15. Eles não foram corrigidos nem testados por estarem fora do escopo. A correção seria a mesma, reutilizando _sessao_autenticada e _quartos_ocupados.

10. Interpretação da cobertura e limitações

- **Cobertura não é ausência de defeitos.** Na etapa 2, o código original tinha 100% de comandos cobertos e 33 casos falhando. Em show_rooms, por exemplo, a linha do predicado tautológico era executada em todos os casos da consulta e sempre errava. Cobertura mede o que foi *executado*, não o que foi *verificado*.

Quem revela o defeito é o oráculo.

- **Comandos × desvios.** Depois da etapa 1, a cobertura de comandos (90,3%) era 7 pontos maior que a de desvios (83,3%). Os desvios 205→204 e 208→205 estavam em linhas executadas, mas um dos lados da decisão nunca ocorreu. Esse lado era o caso "vínculo de outro quarto", que depois levou ao DEF-15.
- **Desvio inviável.** O 230→229 do original não pode ser coberto porque as consultas retornam as reservas em ordem de chave primária. A correção eliminou esse laço.
- **Global × recorte.** No `views.py` inteiro, a cobertura final é de 59,0%. O restante corresponde a rotas fora do escopo. É uma lacuna declarada, não um defeito da suíte.
- **100% de desvios ≠ suíte forte.** A etapa 3 mostrou 5 mutantes não equivalentes vivos com 100% de desvios.
- **Limitações.**
 - Os testes usam o cliente de teste do Flask e SQLite em memória: não há automação de navegador, carga ou concorrência real (duas reservas simultâneas do mesmo quarto não foram testadas).
 - A proteção CSRF não é validada pelas rotas do original; a correção de DEF-10 (POST) reduz o risco, mas não o elimina.
 - As senhas são gravadas em texto plano (`models.py`), fora do recorte.
 - Alguns oráculos são convenções declaradas (seção 3), não requisitos escritos pela autora.
 - O Cosmic Ray foi aplicado só ao recorte.

11. Conclusões e lições aprendidas

1. **A técnica funcional encontrou a maior parte dos defeitos** (16 de 19) sem olhar o código, porque as classes inválidas e os limites foram tratados um a um. O limite "entrada hoje" e as estadias adjacentes expuseram defeitos que valores típicos não mostrariam.
2. **A técnica estrutural encontrou o que a especificação não sugere:** estado global compartilhado, remoção durante a iteração e produto cartesiano. Ela também mostrou que um defeito (DEF-15) pode ficar **maskado** por outro (DEF-01), só aparecendo depois da primeira correção.
3. **A mutação avaliou os próprios testes.** Com 100% de desvios, a suíte ainda tinha fragilidades concretas (dados homogêneos, oráculos assimétricos), corrigidas com 5 casos pequenos.
4. **Corrigir antes de mutar é indispensável.** Mutar o original, com 33 testes falhando, invalidaria o score, porque um mutante que "corrige" um defeito seria contado como vivo ou morto por acaso.
5. **Dificuldades:**
 - dependências de 2019 incompatíveis com o Python atual, resolvidas com ambientes separados e `compat.py`;
 - a execução do Cosmic Ray no Windows (caminho relativo do interpretador);
 - a decisão sobre oráculos não escritos, resolvida declarando convenções.

12. Reprodução

```
git clone https://github.com/gabrielfjm/Hotel_Management_System ; cd Hotel_Management_System
./setup_local.ps1          # ambientes .venv (Python 3.9) e .venv-mutation (Python 3.12) via uv
./etapas.ps1              # métricas, etapas 1-2 (original), suíte corrigida, mutação inicial/final, suíte final
./executar.ps1            # sistema em http://127.0.0.1:5000 (ana@example.test / senha123)
```

Comandos isolados:

- `SUT_VERSAO=original pytest -m funcional -rxX`
- `pytest --cov=hotel.views --cov-branch --cov-report=term-missing`
- `.venv-mutation/Scripts/python.exe scripts/mutacao.py final`
- `git diff sut-original sut-corrigido -- hotel`

Apêndice A: rastreabilidade classe → casos

Classe	Req.	Condição	Tipo	Descrição	Casos funcionais	Outros casos
CE-01	REQ-01	C01 Sessão	válida	Usuário autenticado	CT-001	CT-050
CE-02	REQ-01	C01 Sessão	inválida	Sem sessão (nunca fez login) ou sessão encerrada	CT-014	CT-048
CE-03	REQ-01	C02 Data de entrada	válida	Hoje ou data futura	CT-001, CT-011, CT-012	—
CE-04	REQ-01	C02 Data de entrada	inválida	Data passada	CT-013	—
CE-05	REQ-01	C03 Duração (saída – entrada)	válida	1 noite ou mais	CT-001, CT-008	CT-059
CE-06	REQ-01	C03 Duração (saída – entrada)	inválida	0 noites (saída = entrada)	CT-009	—
CE-07	REQ-01	C03 Duração (saída – entrada)	inválida	Negativa (saída antes da entrada)	CT-010	—
CE-08	REQ-01	C04 N° de hóspedes	válida	Inteiro de 1 até a capacidade somada dos quartos	CT-001, CT-002, CT-003, CT-006, CT-060	—
CE-09	REQ-01	C04 N° de hóspedes	inválida	Menor que 1	CT-007	—
CE-10	REQ-01	C04 N° de hóspedes	inválida	Maior que a capacidade somada	CT-004, CT-005	—
CE-45	REQ-01	C04 N° de hóspedes	inválida	Não inteiro (texto)	CT-052	—
CE-11	REQ-01	C05 Formato dos quartos	válida	Números inteiros separados por vírgula	CT-001	—

Classe	Req.	Condição	Tipo	Descrição	Casos funcionais	Outros casos
CE-12	REQ-01	C05 Formato dos quartos	inválida	Texto não numérico	CT-015	—
CE-13	REQ-01	C06 Existência dos quartos	válida	Todos os quartos existem	CT-001	CT-056
CE-14	REQ-01	C06 Existência dos quartos	inválida	Algum quarto não existe	CT-016	—
CE-15	REQ-01	C07 Repetição de quartos	válida	Sem repetição	CT-001	—
CE-16	REQ-01	C07 Repetição de quartos	inválida	Quarto repetido	CT-017	—
CE-17	REQ-01	C08 Ocupação no período	válida	Nenhuma reserva do quarto com interseção (inclui estadias adjacentes)	CT-001, CT-021, CT-022, CT-023, CT-061, CT-062	CT-044, CT-045, CT-055
CE-18	REQ-01	C08 Ocupação no período	inválida	Reserva existente do quarto com interseção de ao menos uma noite	CT-018, CT-019, CT-020	—
CE-19	REQ-01	C09 Quantidade de quartos	válida	Um quarto	CT-001	CT-044, CT-059
CE-20	REQ-01	C09 Quantidade de quartos	válida	Vários quartos (custo e capacidade somados)	CT-002, CT-060	CT-056
CE-43	REQ-01	C10 Formato das datas	válida	Datas válidas no formato MM/DD/AAAA	CT-001	—
CE-44	REQ-01	C10 Formato das datas	inválida	Data malformada ou inexistente	CT-051	—
CE-21	REQ-02	C11 Sessão	válida	Usuário autenticado	CT-024, CT-029	CT-057
CE-22	REQ-02	C11 Sessão	inválida	Sem sessão ou sessão encerrada	CT-025	CT-048
CE-23	REQ-02	C12 Existência da reserva	válida	Reserva existente	CT-024, CT-029	CT-057
CE-24	REQ-02	C12 Existência da reserva	inválida	Identificador inexistente	CT-026	—
CE-25	REQ-02	C13 Titularidade	válida	Reserva do próprio usuário	CT-024, CT-029	CT-057
CE-26	REQ-02	C13 Titularidade	inválida	Reserva de outro usuário	CT-027	CT-058
CE-27	REQ-02	C14 Forma da requisição	válida	Confirmação explícita (POST)	CT-024, CT-029	CT-057
CE-28	REQ-02	C14 Forma da requisição	inválida	Navegação por link (GET)	CT-028	—
CE-29	REQ-02	C15 Registros dependentes	válida	Reserva sem pagamento	CT-024	—
CE-30	REQ-02	C15 Registros dependentes	válida	Reserva com pagamento registrado	CT-029	—
CE-31	REQ-03	C16 Sessão	válida	Usuário autenticado	CT-030, CT-032	CT-049
CE-32	REQ-03	C16 Sessão	inválida	Sem sessão ou sessão encerrada	CT-031	CT-048
CE-33	REQ-03	C17 Filtro	válida	Sem filtro: lista geral de quartos	CT-030	CT-046
CE-34	REQ-03	C17 Filtro	válida	Com período e hóspedes informados	CT-032	—
CE-35	REQ-03	C18 Período	válida	Entrada anterior à saída (1 noite ou mais)	CT-032, CT-043	—
CE-36	REQ-03	C18 Período	inválida	Entrada igual ou posterior à saída	CT-037, CT-038	—
CE-46	REQ-03	C19 Formato das datas	válida	Datas válidas no formato MM/DD/AAAA	CT-032	—
CE-47	REQ-03	C19 Formato das datas	inválida	Data malformada ou inexistente	CT-053	—
CE-37	REQ-03	C20 N° de hóspedes	válida	Inteiro maior ou igual a 1	CT-032, CT-042	—
CE-38	REQ-03	C20 N° de hóspedes	inválida	Menor que 1	CT-039	—
CE-48	REQ-03	C20 N° de hóspedes	inválida	Não inteiro (texto)	CT-054	—
CE-39	REQ-03	C21 Ocupação de cada quarto	válida	Livre no período: exibido	CT-032, CT-033, CT-034, CT-035, CT-063, CT-064	—
CE-40	REQ-03	C21 Ocupação de cada quarto	válida	Ocupado no período: omitido	CT-032, CT-036	CT-047
CE-41	REQ-03	C22 Capacidade de cada quarto	válida	Capacidade ≥ hóspedes: exibido	CT-032, CT-040, CT-042	—
CE-42	REQ-03	C22 Capacidade de cada quarto	válida	Capacidade < hóspedes: omitido	CT-041	—